

Data Analysis and Visualization

PART II:

Advanced Visualization and Creating Dashboards

Lecturer: Diu Tran, PhD

Department of Computer Science
Faculty of Information Technology
Ton Duc Thang University

Ho Chi Minh, Week 3, 2025

Content

- 1 Revise week 2
- 2 Advanced Data Visualization
 - Waffle Charts
 - Word Cloud
 - Seaborn Library
 - Folium Library
 - Choropleth Maps
- 3 Lab 3

Revise week 2

Exploring and pre-processing data using Pandas

Preprocessing_Exploring_with_Pandas.ipynb

Skills:

- Read, view, save dataset
- Uncover some information of data: data shape, column names, indices, data types
- Check null values
- Calculate new features which can be used for investigating
- Extract statistical information of the dataset
- Select columns, select rows
- Filter the dataframe based on a condition
- Sorting Values of a dataframe

Revise week 2

Exploring and pre-processing data using Pandas

Cheat Sheet : Data Preprocessing Tasks in Pandas

Task	Syntax	Description	Example
Load CSV data	<code>pd.read_csv('filename.csv')</code>	Read data from a CSV file into a Pandas DataFrame	<code>df_canpd.read_csv('data.csv')</code>
Handling Missing Values	<code>df.dropna()</code>	Drop rows with missing values	<code>df_can.dropna()</code>
	<code>df.fillna(value)</code>	Fill missing values with a specified value	<code>df_can.fillna(0)</code>
Removing Duplicates	<code>df.drop_duplicates()</code>	Remove duplicate rows	<code>df_can.drop_duplicates()</code>
Renaming Columns	<code>df.rename(columns={'old_name': 'new_name'})</code>	Rename one or more columns	<code>df_can.rename(columns={'Age': 'Years'})</code>
Selecting Columns	<code>df['column_name']</code> or <code>df.column_name</code>	Select a single column	<code>df_can.Age</code> or <code>df_can['Age']</code>
	<code>df[['col1', 'col2']]</code>	Select multiple columns	<code>df_can[['Name', 'Age']]</code>
Filtering Rows	<code>df[df['column'] > value]</code>	Filter rows based on a condition	<code>df_can[df_can['Age'] > 30]</code>
Applying Functions to Columns	<code>df['column'].apply(function_name)</code>	Apply a function to transform values in a column	<code>df_can['Age'].apply(lambda x: x + 1)</code>
Creating New Columns	<code>df['new_column'] = expression</code>	Create a new column with values derived from existing ones	<code>df_can['Total'] = df_can['Quantity'] * df_can['Price']</code>
Grouping and Aggregating	<code>df.groupby('column').agg({'col1': 'sum', 'col2': 'mean'})</code>	Group rows by a column and apply aggregate functions	<code>df_can.groupby('Category').agg({'Total': 'mean'})</code>
Sorting Rows	<code>df.sort_values('column', ascending=True/False)</code>	Sort rows based on a column	<code>df_can.sort_values('Date', ascending=True)</code>
Displaying First n Rows	<code>df.head(n)</code>	Show the first n rows of the DataFrame	<code>df_can.head(3)</code>
Displaying Last n Rows	<code>df.tail(n)</code>	Show the last n rows of the DataFrame	<code>df_can.tail(3)</code>
Checking for Null Values	<code>df.isnull()</code>	Check for null values in the DataFrame	<code>df_can.isnull()</code>
Selecting Rows by Index	<code>df.iloc[index]</code>	Select rows based on integer index	<code>df_can.iloc[3]</code>
	<code>df.iloc[start:end]</code>	Select rows in a specified range	<code>df_can.iloc[2:5]</code>
Selecting Rows by Label	<code>df.loc[label]</code>	Select rows based on label/index name	<code>df_can.loc['label']</code>
	<code>df.loc[start:end]</code>	Select rows in a specified label/index range	<code>df_can.loc['Age':'Quantity']</code>
Summary Statistics	<code>df.describe()</code>	Generates descriptive statistics for numerical columns	<code>df_can.describe()</code>

Revise week 2: Types of plots

- Line plots
- Area plots
- Histograms
- Bar plots
- Pie charts
- Box plots
- Scatter plots
- Bubble plots

Files:

- `Data_visualization1_Matplotlib.ipynb`
- `Data_visualization2_Matplotlib.ipynb`

Section: Advanced Data Visualization

- Waffle Charts
- Word Cloud
- Choropleth Maps
- Treemaps and Pivot Charts
(`Treemaps_and_Pivot_Charts.ipynb`)

Python libraries:

- *pywaffle* Library
- Seaborn Library
- Folium Library

Waffle Charts

- Represent categorical data in the form of square tiles or cells
- Display the proportion or percentage of different categories
- Simplifies data for all types of audiences



Waffle Charts: Use cases

- Market share analysis: visualize market share data showing the proportion of companies or products within a specific market
- Demographic representation: display demographic data such as age groups or ethnicities within a population
- Project progress tracking: represent completion status of tasks or milestones
- Budget allocation: demonstrate allocation of budgetary resources across categories
- Survey responses: summarize survey responses, displaying the distribution of answers to multiple choice questions
- Election results: provide a clear visualization of voting outcomes
- Product sales analysis: illustrate product sales by representing the distribution of sales across categories or regions

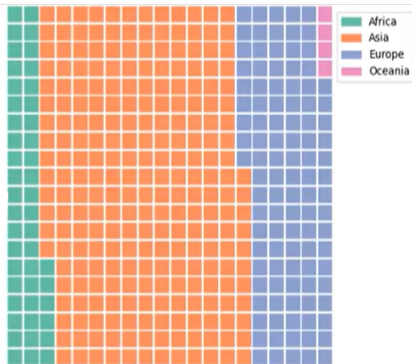
pywaffle Library in Python

```
import matplotlib.pyplot as plt
from pywaffle import Waffle

# Create data for the waffle chart
data = df_dsn[['Africa', 'Asia', 'Europe', 'Oceania']]

# Set up the waffle chart figure
fig = plt.figure(
    FigureClass=Waffle,
    rows=20,
    columns=20,
    values=data,
    legend={'labels':['Africa', 'Asia', 'Europe', 'Oceania'],
           'loc': 'upper left', 'bbox_to_anchor': (1, 1)})

# Display the chart
plt.show()
```



File: WaffleCharts_WordClouds_RegressionPlots.ipynb

Word Cloud

- Is a popular data visualization method to visually present textual data in an engaging and informative manner
- Presents a concise summary of the textual content by providing a visual overview of the most commonly used words within a given text or collection of documents
- Depicts of the importance of different words in a body of text.

File: `WaffleCharts_WordClouds_RegressionPlots.ipynb`

Word Cloud: Use cases

You can use word cloud for

- Social media analysis: extract and visualize popular topics or sentiments from social media conversations or hashtags
- Customer feedback analysis: summarize customer reviews or feedback
- Content analysis: analyze textual content such as articles, blogs, or research papers to uncover prevalent keywords or themes
- Market research: analyze survey responses, interviews
- Resume or job description analysis: highlight important skills or keywords in resumes or job descriptions to assess their relevance

Seaborn Library

- Seaborn is Python visualization library based on Matplotlib
- Seaborn offers built-in themes and color palettes to improve your plot visual
- Seaborn makes creating plots very efficient, you can generate plots with code that is five times less than with Matplotlib
- Seaborn integrates well with statistical libraries such as NumPy and SciPy, allowing you to easily combine statistical analysis with visualizations.

Seaborn: special plots

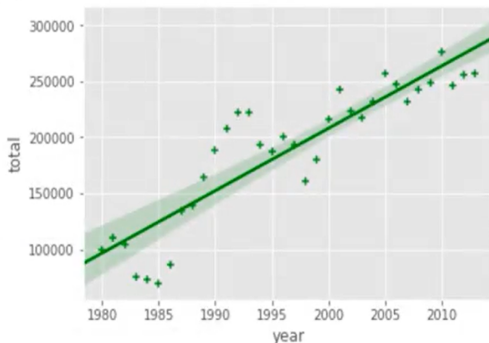
- Regression plots
- Distribution plots
- Categorical plots
- Creating Scatter plots with Regression lines

Regression plots

df_total

Year	Total
1980	99137
1981	110563
1982	104271
1983	75550
1984	73417
.	.
.	.

```
import seaborn as sns
ax = sns.regplot(x='year', y='total', data=df_tot,
                 color='green', marker='+')
```



File: WaffleCharts_WordClouds_RegressionPlots.ipynb

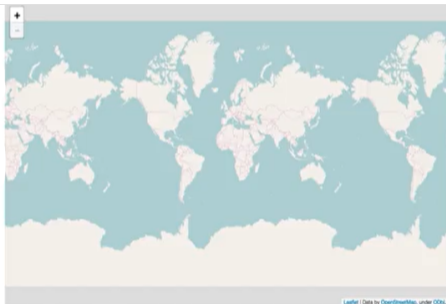
What is Folium?

- Folium is a powerful data visualization library in Python that was built primarily to help people visualize geospatial data
- With Folium, you can create a map of any location in the world using latitude and longitude values. You can also create a map and superimpose markers and clusters on top of the map for interesting visualizations
- You can also create maps of different styles, such as street level maps, stamen maps, and a couple of others

Creating a world map

```
#import library  
import folium
```

```
# define the world map  
world_map = folium.Map()  
  
# display world map  
world_map
```



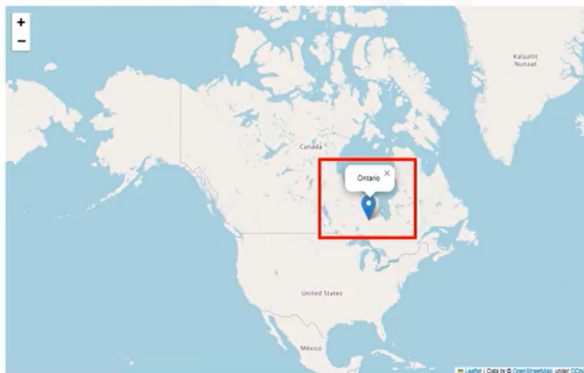
Creating a map of a country

```
# create a Stamen Toner map of  
# the world centered around Canada  
world_map = folium.Map(  
    location=[56.130, -106.35],  
    zoom_start=4,  
    tiles='Stamen Terrain'  
)  
  
# display map  
world_map
```



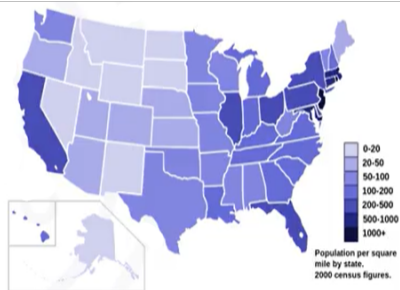
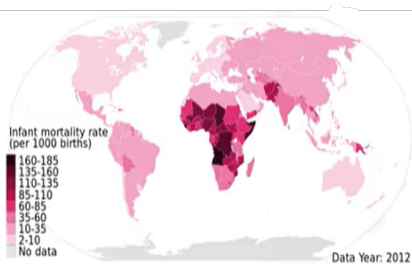
Maps with makers

```
# Add a marker for Ontario province  
folium.Marker(location=[51.2538, -85.3232], popup='Ontario').add_to(canada_map)
```



Jupyter notebook: `Creating_maps_visualizing_geospat.ipynb`

Choropleth Maps



Choropleth Maps

- A choropleth map is a thematic map in which areas are shaded or patterned in proportion to the measurement of the statistical variable
- When creating a choropleth map, Folium requires a GeoJSON file that includes geospatial data of the region
- The Mapbox Bright Tileset displays the name of every country when used on a map.

GeoJSON file

```
{
  "type": "FeatureCollection",
  "features": [{
    "type": "Feature",
    "properties": {
      "name": "Brunei"
    },
    "geometry": {
      "type": "Polygon",
      "coordinates": [
        [
          [114.204017, 4.525874], [114.599961, 4.900011], [115.45071, 5.44773],
          [115.4057, 4.955228], [115.347461, 4.316636], [114.869557, 4.348314],
          [114.659596, 4.007637], [114.204017, 4.525874]
        ]
      ]
    },
    "id": "BRN"
  },
```

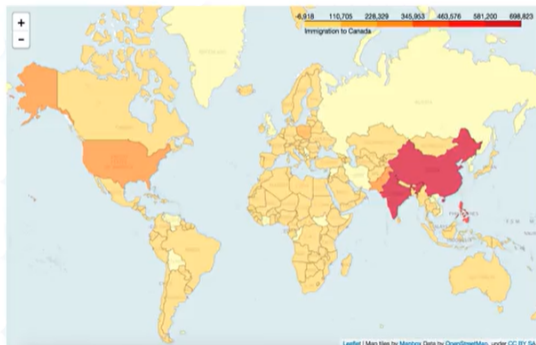
Creating the Choropleth Map

```
# create a plain world map
world_map = folium.Map(
    zoom_start=2,
    tiles='Mapbox Bright'
)

## geojson file
world_geo = r'world_countries.json'

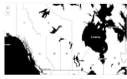
# generate choropleth map using the total
# population of each country to Canada from
# 1980 to 2013
world_map.choropleth(
    geo_path=world_geo,
    data=df_canada,
    columns=['Country', 'Total'],
    key_on='feature.properties.name',
    fill_color='YlOrRd',
    legend_name='Immigration to Canada'
)

# display map
world_map
```

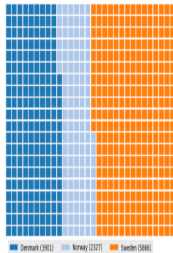


Jupyter notebook: `Creating_maps_visualizing_geospat.ipynb`

Cheat Sheet : Maps

Function	Description	Syntax	Example	Visual
Folium				
Map	Create a map object with specified center coordinates and zoom level.	<code>folium.Map(location=[lat, lon], zoom_start=n)</code>	<pre>world_map = folium.Map() canada = folium.Map(location=[56.130, -106.35], zoom_start=4)</pre>	
Marker	Add a marker to the map with custom icon, popup, and tiles Tiles as Stamen Toner	<code>folium.Marker(location=[lat, lon], popup='Marker Popup', tiles='Stamen Toner').add_to(map)</code>	<code>folium.Marker(location=[556.130, -106.35], tooltip='Marker', tiles='Stamen Toner').add_to(world_map)</code>	
	Tiles as Stamen Terrain	<code>folium.Marker(location=[lat, lon], popup='Marker Popup', tiles='Stamen Terrain').add_to(map)</code>	<code>folium.Marker(location=[556.130, -106.35], tooltip='Marker', tiles='Stamen Terrain').add_to(world_map)</code>	
Circle	Add a circle to the map with specified radius, color, and fill opacity.	<code>folium.features.CircleMarker(location=[lat, lon], radius=n, color='red', fill_opacity=n).add_to(map)</code>	<code>folium.features.CircleMarker(location=[56.130, -106.35], radius=1000, color='red', fill_opacity=0.5).add_to(world_map)</code>	
Choropleth	Create a choropleth map based on a GeoJSON file and a specified data column.	<code>folium.Choropleth(geo_data='path/to/geojson_file', data=df, columns=['region', 'value.column'], key_on='feature.properties.id', fill_color='YlGnBu', fill_opacity=0.7, line_opacity=0.2, legend_name='Legend').add_to(map)</code>	<code>world_map.choropleth(geo_data=world_geo, data=df_can, columns=['Country', 'Total'], key_on='feature.properties.name', fill_color='YlOrRd', fill_opacity=0.7, line_opacity=0.2, legend_name='Immigration to Canada')</code>	

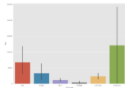
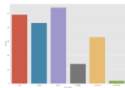
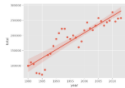
Cheat Sheet : Waffles

PyWaffle				
Waffle	Create a waffle chart based on values and categories.	<pre>plt.figure(FigureClass = Waffle, rows = 20, columns = 30, values = values) waffle_chart = waffle.Waffle(values=[value1, value2, ...], rows=n, columns=n)</pre>	<pre>plt.figure(FigureClass = Waffle, rows = 20, columns = 30, values = df.dsn['Total'], cmap_name = 'tab20', legend = {'labels': label, 'loc': 'lower left', 'bbox_to_anchor': (0, -0.1), 'ncol': 3})</pre>	
Legend	Add a legend to the waffle chart.	<code>waffle_chart.legend(loc='upper left', bbox_to_anchor=(1, 1))</code>		
Title	Add a title to the waffle chart.	<code>waffle_chart.set_title('Waffle Chart Title')</code>		
Labels	Add labels to the waffle chart.	<code>waffle_chart.set_labels(['Label 1', 'Label 2', ...])</code>		

Cheat Sheet: WordCloud

WordCloud				
WordCloud	Create a word cloud object based on text data.	wordcloud = WordCloud().generate(text_data)	<pre>alice_wc = WordCloud(background_color='white', max_words=2000, mask=alice_mask, stopwords=stopwords) alice_wc.generate(alice_novel) plt.imshow(alice_wc, interpolation='bilinear')</pre>	
Generate	Generate the word cloud based on the text data.	wordcloud.generate(text_data)		
Display	Display the word cloud using matplotlib or other plotting libraries.	plt.imshow(wordcloud, interpolation='bilinear')		
Options	Set various options for the word cloud, such as font, colors, mask, and stopwords.	wordcloud = WordCloud(font_path='path/to/font_file', background_color='white', colormap='Blues', mask=mask_image, stopwords=stopwords).generate(text_data)		

Cheat Sheet : Seaborn

Seaborn				
barplot	Create a bar plot to visualize the relationship between a categorical variable and a numeric variable.	<code>sns.barplot(x='x_variable', y='y_variable', data=dataframe)</code>	<code>sns.barplot(x='Continent', y='Total', data=df_can1)</code>	
countplot	Create a count plot to display the frequency of each category in a categorical variable.	<code>sns.countplot(x='category', data=dataframe)</code>	<code>sns.countplot(x='Continent', data=df_can)</code>	
regplot	Create a scatter plot with a linear regression line to visualize the relationship between two numeric variables.	<code>sns.regplot(x='x_variable', y='y_variable', data=dataframe)</code>	<code>sns.regplot(x='year', y='total', data=df_tot)</code>	

Lab 3

Using the datasets that you collected from Lab 1:

- 1 Practice wrangle data with Pandas: data pre-processing, data summarizing. Reference
`Preprocessing_Exploring_with_Pandas.ipynb` (Lab 2).
Cleaning datasets (pre-processing datasets) before visualization as in example
`Preprocessing_Exploring_with_Pandas.ipynb`.
- 2 Data visualization using Matplotlib. Reference
`Data_visualization1_Matplotlib.ipynb` and
`Data_visualization2_Matplotlib.ipynb` (Lab 2)
- 3 Data visualization: practice waffle charts, word cloud, choropleth maps. Reference
`Creating_maps_visualizing_geospat.ipynb` and
`WaffleCharts_WordClouds_RegressionPlots.ipynb` (Lab 3)